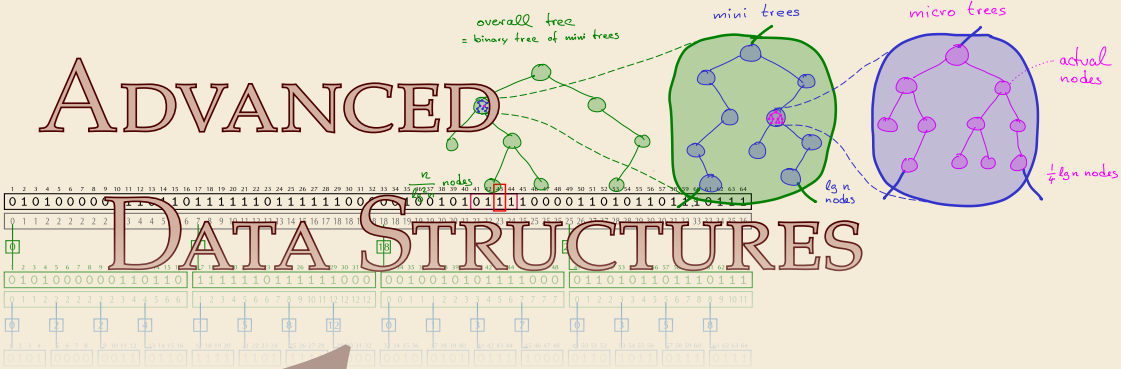
Advanced Data Structures



1 Randomized Trees

Advanced Data Structures · Summer 2026

Prof. Dr. Sebastian Wild

CS650 (Summer 2026)
Philipps-Universität Marburg
version 2026-04-13 15:56

Outline

1 Randomized Trees

1.1 Recap: Sorted Dictionary and BSTs
1.2 What's wrong with BBSTs?
1.3 Random BSTs
1.4 Treaps
1.5 Updates in Treaps
1.6 Insertion at Root
1.7 Randomized Binary Search Trees
1.8 Skip Lists
1.9 Jumplists

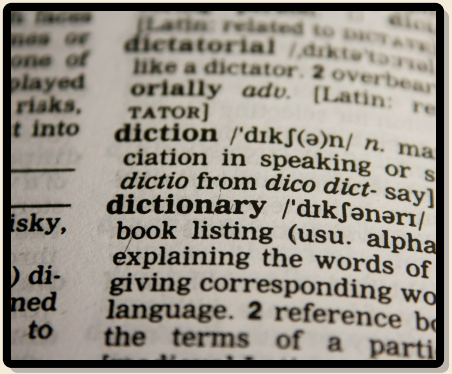
1.1 Recap: Sorted Dictionary and BSTs

Symbol table ADT

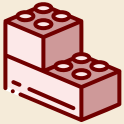
Java: `java.util.Map<K,V>`

Symbol table / Dictionary / Map / Associative array / key-value store:

Python dict `{k:v}`



- `put(`k`,`v`)` *Python* dict: `d[`k`] = `v
  Put key-value pair (k, v) into table
- `get(`k`)` *Python* dict: `d[`k`]`
  Return value associated with key k
- `delete(`k`)` *Python* dict: `del d[`k`]`
  Remove key k (any associated value) form table
- `contains(`k`)` *Python* dict: k `in d`
  Returns whether the table has a value for key k
- `isEmpty(), size()`
- `create()`

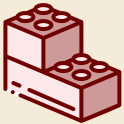


Most fundamental building block in computer science.

(Every programming library has a symbol table implementation.)

Ordered symbol tables

- `min()`, `max()`
  Return the smallest resp. largest key in the ST
- `floor(`x`)`, $\lfloor x \rfloor = \mathbb{Z}.$`floor(`$x$`)`
  Return largest key k in ST with $k \leq x$.
- `ceiling(`x`)`
  Return smallest key k in ST with $k \geq x$.
- `rank(`x`)`
  Return the number of keys k in ST $k < x$.
- `select(`i`)`
  Return the ith smallest key in ST (zero-based, i. e., $i \in [0..n)$)



With select, we can simulate access as in a truly dynamic array!

(Might not need any keys at all then!)

Binary search trees

Binary search trees (BSTs) $\approx$ dynamic sorted array

- binary tree
  - Each node has left and right child
  - Either can be empty (`null`)
- Keys satisfy ***search-tree property***

$$\boxed{\text{all keys in left subtree} \leq \text{root key} \leq \text{all keys in right subtree}}$$

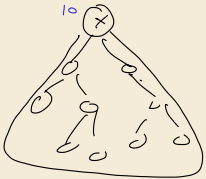
Binary search trees

Binary search trees (BSTs) $\approx$ dynamic sorted array

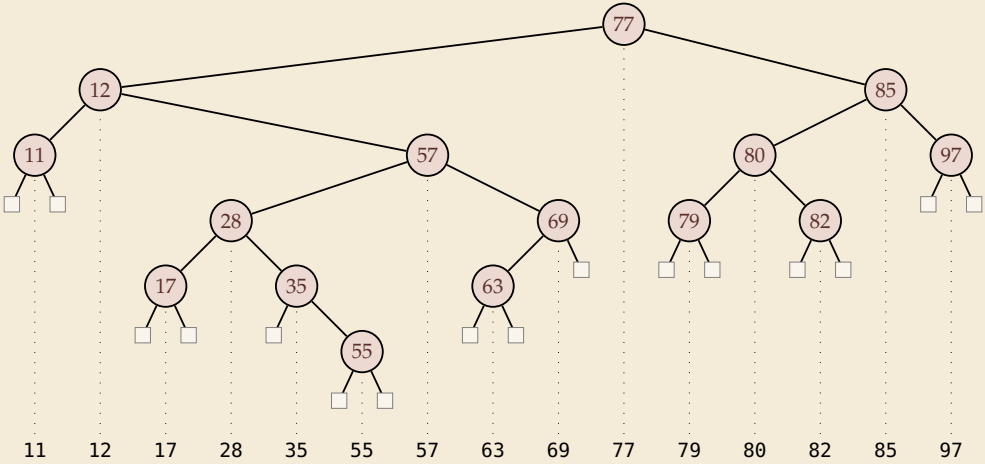
- binary tree
  - Each node has left and right child
  - Either can be empty (`null`)
- Keys satisfy ***search-tree property***

  all keys in left subtree $\leq$ root key $\leq$ all keys in right subtree

- Standard trick: ***Augmented BSTs***
  - Each node stores the **size** of its **subtree**
  - Easy to maintain upon updates
  - $\rightsquigarrow$ Allows to answer all ordered-symbol-table operations

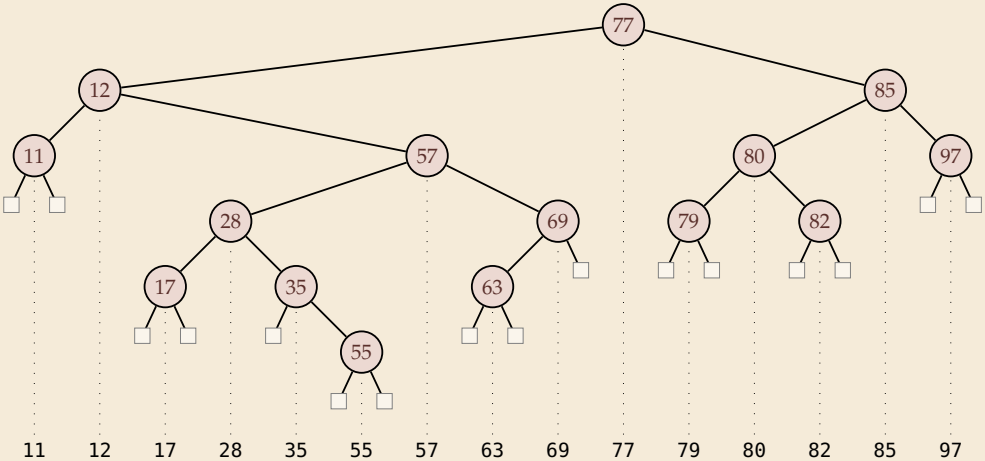


BST example & find



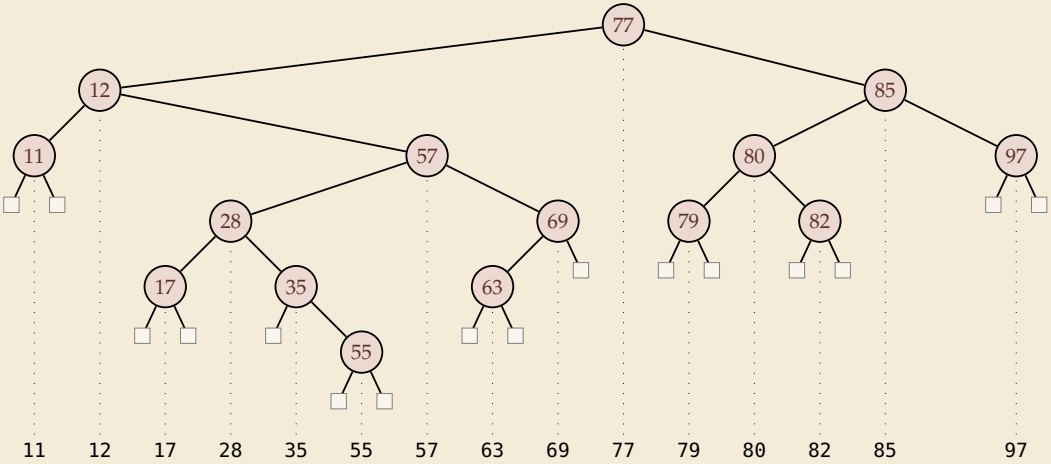
BST insert

Example: Insert 88



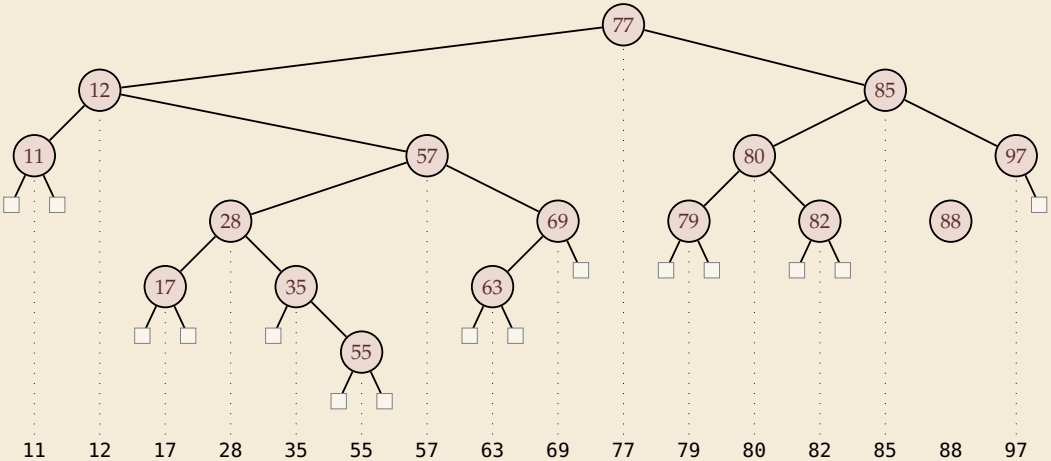
BST insert

Example: Insert 88



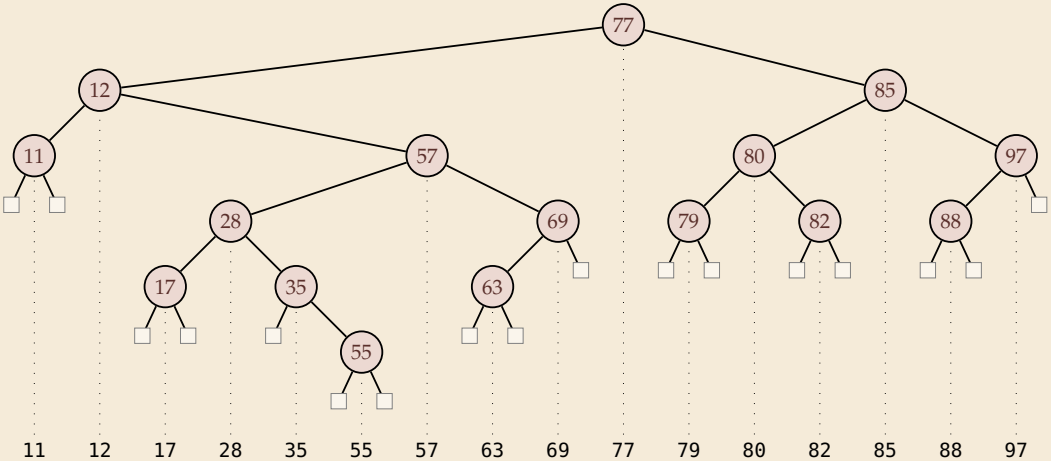
BST insert

Example: Insert 88



BST insert

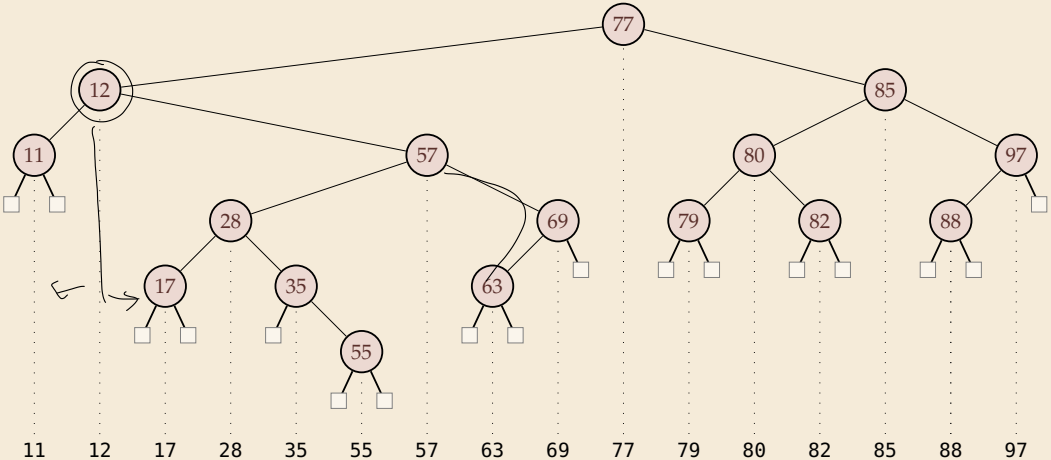
Example: Insert 88



BST delete

Hibbard's deletion

- Easy case: remove leaf, e. g., 11 ⇝ replace by `null`
- Medium case: remove unary, e. g., 69 ⇝ replace by unique child
- Hard case: remove binary, e. g., 85 ⇝ swap with predecessor, recurse



Unbalanced Binary Search Trees

Operation	Running Time
`construct(`$A[1..n]$`)`	$O(n\boldsymbol{h})$
`put(`k`,`v`)`	$O(\boldsymbol{h})$
`get(`k`)`	$O(\boldsymbol{h})$
`delete(`k`)`	$O(\boldsymbol{h})$
`contains(`k`)`	$O(\boldsymbol{h})$
`isEmpty()`	$O(1)$
`size()`	$O(1)$
`min(),max()`	$O(1)$ (if stored)
`floor(`x`),ceiling(`x`)`	$O(\boldsymbol{h})$
`rank(`x`),select(`i`)`	$O(\boldsymbol{h})$

$h =$ *height* of the BST

$1 - 2^{-n}$

- Height h of unbalanced BST depends on insertion order
- satisfies $\lg n \leq h \leq n$

$\lg = \log_2$

$\text{Prob} = 1 - o(1)$

- For **random** insertions, $h = O(\log n)$ in expectation and with high probability

Balanced Binary Search Tree

Balanced binary search trees (BBSTs):

- imposes shape invariant that guarantees $O(\log n)$ height
- adds rules to restore invariant after updates

Balanced Binary Search Tree

Balanced binary search trees (BBSTs):

- imposes shape invariant that guarantees $O(\log n)$ height
- adds rules to restore invariant after updates
- **Classical examples**
  - *AVL trees* (height-balanced trees)
  - *red-black trees*

  every undergrad data-structures module I know includes at least one of these two
  - *weight-balanced trees* (BB[α] trees)

Balanced Binary Search Tree

Balanced binary search trees (BBSTs):

- imposes shape invariant that guarantees $O(\log n)$ height
- adds rules to restore invariant after updates
- **Classical examples**
  - *AVL trees* (height-balanced trees)
  - *red-black trees*

  every undergrad data-structures module I know includes at least one of these two
  - *weight-balanced trees* (BB[α] trees)
- **Properties**
  - All of them guarantee $h = O(\log n)$ at all times (constants differ!)
  - All of them work with *rotations* along the search path

  $\rightsquigarrow$ $O(\log n)$ extra cost for maintaining balance
  - Further guarantees specific to rule known

    e. g., BB[α] trees only rotate a node again after a linear number of updates to its subtree

Balanced Binary Search Tree

Balanced binary search trees (BBSTs):

- imposes shape invariant that guarantees $O(\log n)$ height
- adds rules to restore invariant after updates
- **Classical examples**
  - *AVL trees* (height-balanced trees)
  - *red-black trees*

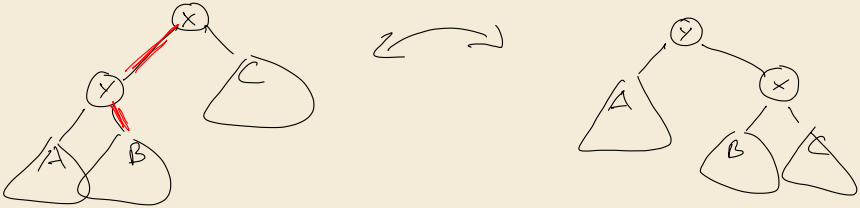
  every undergrad data-structures module I know includes at least one of these two
  - *weight-balanced trees* (BB[α] trees)
- **Properties**
  - All of them guarantee $h = O(\log n)$ at all times (constants differ!)
  - All of them work with *rotations* along the search path

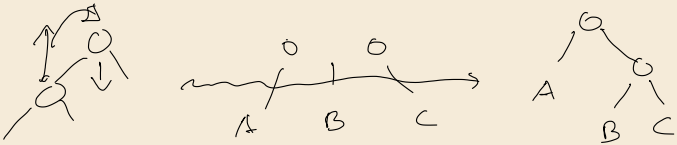
  $\rightsquigarrow$ $O(\log n)$ extra cost for maintaining balance
  - Further guarantees specific to rule known

    e. g., BB[α] trees only rotate a node again after a linear number of updates to its subtree

OPEN: *What is the exact average cost (constant in front of $\lg n$) of insertion in AVL trees / red-black trees?*

A Primitive for BSTs: Rotations





1.2 What's wrong with BBSTs?

Red-Black Tree Implementation

`RedBlackTree` from Sedgewick & Wayne (excerpt) `algs4.cs.princeton.edu/code`

- showing only the bare-bones core: `put` (insert) and `delete`
- no rank-based operations
- full class has 750 lines (and this is a good and compact implementation!)

```
public class RedBlackBST<Key extends Comparable<Key>, Value> {
    public void put(Key key, Value val) {
        root = put(root, key, val);
        root.color = BLACK;
    }

    private Node put(Node h, Key key, Value val) {
        if (h == null) return new Node(key, val, RED, 1);

        int cmp = key.compareTo(h.key);
        if      (cmp < 0) h.left  = put(h.left,  key, val);
        else if (cmp > 0) h.right = put(h.right, key, val);
        else              h.val   = val;

        // fix-up any right-leaning links
        if (isRed(h.right) && !isRed(h.left))      h = rotateLeft(h);
        if (isRed(h.left)  &&  isRed(h.left.left)) h = rotateRight(h);
        if (isRed(h.left)  &&  isRed(h.right))     flipColors(h);
        h.size = size(h.left) + size(h.right) + 1;

        return h;
    }

    private Node deleteMin(Node h) {
        if (h.left == null)
            return null;

        if (!isRed(h.left) && !isRed(h.left.left))
            h = moveRedLeft(h);

        h.left = deleteMin(h.left);
        return balance(h);
    }
```

```
    private Node deleteMax(Node h) {
        if (isRed(h.left))
            h = rotateRight(h);

        if (h.right == null)
            return null;

        if (!isRed(h.right) && !isRed(h.right.left))
            h = moveRedRight(h);

        h.right = deleteMax(h.right);

        return balance(h);
    }

    private Node delete(Node h, Key key) {
        // assert get(h, key) != null;

        if (key.compareTo(h.key) < 0)  {
            if (!isRed(h.left) && !isRed(h.left.left))
                h = moveRedLeft(h);
            h.left = delete(h.left, key);
        }
        else {
            if (isRed(h.left))
                h = rotateRight(h);
            if (key.compareTo(h.key) == 0 && (h.right == null))
                return null;
            if (!isRed(h.right) && !isRed(h.right.left))
                h = moveRedRight(h);
            if (key.compareTo(h.key) == 0) {
                Node x = min(h.right);
                h.key = x.key;
```

```
                h.val = x.val;
                // h.val = get(h.right, min(h.right).key);
                // h.key = min(h.right).key;
                h.right = deleteMin(h.right);
            }
            else h.right = delete(h.right, key);
        }
        return balance(h);
    }

    private Node rotateRight(Node h) {
        assert (h != null) && isRed(h.left);
        Node x = h.left;
        h.left = x.right;
        x.right = h;
        x.color = h.color;
        h.color = RED;
        x.size = h.size;
        h.size = size(h.left) + size(h.right) + 1;
        return x;
    }

    private Node rotateLeft(Node h) {
        assert (h != null) && isRed(h.right);
        Node x = h.right;
        h.right = x.left;
        x.left = h;
        x.color = h.color;
        h.color = RED;
        x.size = h.size;
        h.size = size(h.left) + size(h.right) + 1;
        return x;
    }
```

```
    private void flipColors(Node h) {
        h.color = !h.color;
        h.left.color = !h.left.color;
        h.right.color = !h.right.color;
    }

    private Node moveRedLeft(Node h) {
        flipColors(h);
        if (isRed(h.right.left)) {
            h.right = rotateRight(h.right);
            h = rotateLeft(h);
            flipColors(h);
        }
        return h;
    }

    private Node moveRedRight(Node h) {
        flipColors(h);
        if (isRed(h.left.left)) {
            h = rotateRight(h);
            flipColors(h);
        }
        return h;
    }

    private Node balance(Node h) {
        if (isRed(h.right) && !isRed(h.left))    h = rotateLeft(h);
        if (isRed(h.left) && isRed(h.left.left)) h = rotateRight(h);
        if (isRed(h.left) && isRed(h.right))     flipColors(h);

        h.size = size(h.left) + size(h.right) + 1;
        return h;
    }
```

Can we have something simpler?

In this and the next unit, we will look into alternatives to avoid lengthy case distinctions.

Can we have something simpler?

In this and the next unit, we will look into alternatives to avoid lengthy case distinctions.

Disclaimer: It seems that something has to give.

All known (much) more elegant BST alternatives are either *randomized* or *amortized*.

- However, this is often tolerable.
- Complexity of code can also come with running-time penalty;
  if we avoid that, might be an overall speedup.
  (Typically not the case for well-tested library implementations, though.)

Recall: Average-Case Analysis vs. Randomized Algorithms

This unit: *Randomized solutions*.

Recall: Average-Case Analysis vs. Randomized Algorithms

This unit: *Randomized solutions*.

Average-Case Analysis

- algorithm is **deterministic**
  same input, same computation

Randomized Algorithm (here)

- algorithm is **not** deterministic
  same input, potentially different comp.

Recall: Average-Case Analysis vs. Randomized Algorithms

This unit: *Randomized solutions*.

Average-Case Analysis

- algorithm is **deterministic**
  same input, same computation
- input is chosen according to some **probability distribution**

Randomized Algorithm (here)

- algorithm is **not** deterministic
  same input, potentially different comp.
- input is chosen **adversarially** (worst-case inputs)

Recall: Average-Case Analysis vs. Randomized Algorithms

This unit: *Randomized solutions*.

Average-Case Analysis

- algorithm is **deterministic**
  same input, same computation
- input is chosen according to some **probability distribution**
- cost given as expectation over inputs

Randomized Algorithm (here)

- algorithm is **not** deterministic
  same input, potentially different comp.
- input is chosen **adversarially** (worst-case inputs)
- cost given as expectation over random choices of algorithm

Recall: Average-Case Analysis vs. Randomized Algorithms

This unit: *Randomized solutions*.

Average-Case Analysis

- algorithm is **deterministic**
  same input, same computation
- input is chosen according to some **probability distribution**
- cost given as expectation over inputs

Randomized Algorithm (here)

- algorithm is **not** deterministic
  same input, potentially different comp.
- input is chosen **adversarially** (worst-case inputs)
- cost given as expectation over random choices of algorithm

Confusingly enough, the analysis (technique) is often the same!

But: Implications are quite different; randomization is much more versatile and robust.

1.3 Random BSTs

Random BSTs are good (enough)

*Let's first do some wishful thinking: assume we're dealing with **random** inputs only.*

- ⇝ Let's first see if we like the performance in this case.
- If so, will try and see how to **enforce** this randomness later.

Random BSTs are good (enough)

Let's first do some wishful thinking: assume we're dealing with ***random*** *inputs only.*

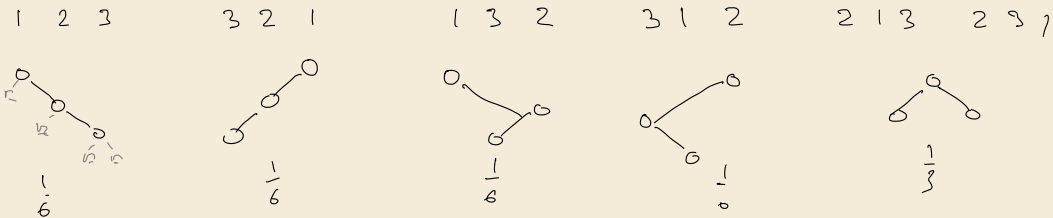
- ⇝ Let's first see if we like the performance in this case.
- ▶ If so, will try and see how to **enforce** this randomness later.

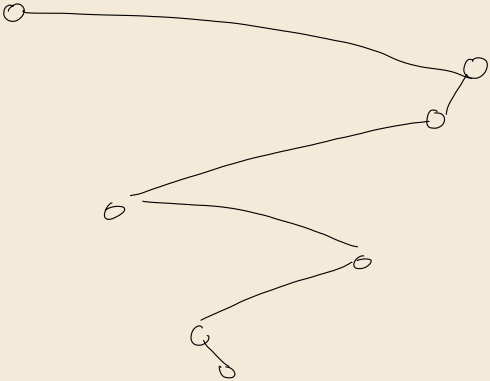
Random here means:

Definition 1.1 (Random BST)

A ***Random BST*** of size n is the (random) shape of an initially empty BST after successively inserting a random permutation of $[n]$.

Example: $n = 3$





$\approx 2^n$ maximally unbalanced trees

#binary trees on n nodes

nth Catalan number

$$= \binom{2n}{n} \cdot \frac{1}{n+1}$$

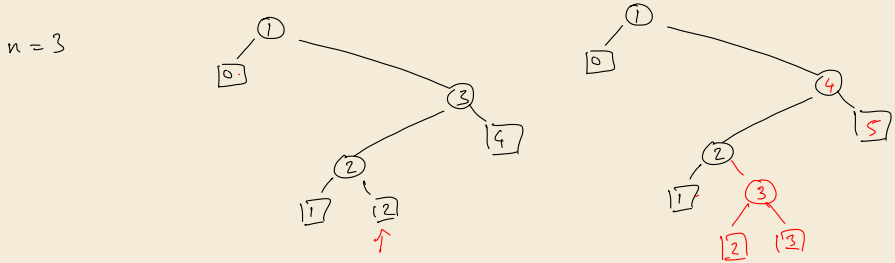
$$\approx 4^n / n^{3/2}$$

$$n! \approx \left(\frac{n}{e}\right)^n$$

Iterative randomness

Lemma 1.2 (Random insertion yields random BST)

Let $n \geq 0$ and T_n a random BST over n keys. Inserting an element equally likely in one of the $n+1$ *gaps* in T_n (external leaves) results in a new BST T_{n+1} that has the same shape as a random BST on $n+1$ keys.



Iterative randomness

Lemma 1.2 (Random insertion yields random BST)

Let $n \geq 0$ and T_n a random BST over n keys. Inserting an element equally likely in one of the $n+1$ *gaps* in T_n (external leaves) results in a new BST T_{n+1} that has the same shape as a random BST on $n+1$ keys.

Proof:

Insertion order in T_{n+1} is in bijection with permutations of $[n+1]$, $[0..n]$ which is in bijection with a permutation of $[n]$ and a *last value* in $[n+1]$, where we increment all values $\geq$ the last value by 1.

$= [1..n+1]$

$= \{1, 2, \ldots, n+1\}$

Example: $1, 2, 4, 7, 6, 5 \mid 3 \;\widehat{=}\; ((1, 2, 3, 6, 5, 4), 3)$

Recall: Events

something we can assign a probability to

$A \in \mathcal{F}$ is called an *event* of $(\Omega, \mathcal{F}, \mathbb{P})$; also a *measurable set*.

Recall: Events

something we can assign a probability to

$A \in \mathcal{F}$ is called an *event* of $(\Omega, \mathcal{F}, \mathbb{P})$; also a *measurable set*.

Basic properties

- $\mathbb{P}\left[\overline{A}\right] = 1 - \mathbb{P}[A]$ counter-probability $\quad (\overline{A} = \Omega \setminus A)$
- $\mathbb{P}\left[\bigcup A_i\right] \leq \sum_i \mathbb{P}[A]$ the *union bound* (a.k.a. Boole's inequality a.k.a. σ-subadditivity)

Recall: Events

something we can assign a probability to

$A \in \mathcal{F}$ is called an *event* of $(\Omega, \mathcal{F}, \mathbb{P})$; also a *measurable set*.

Basic properties

- $\mathbb{P}\left[\overline{A}\right] = 1 - \mathbb{P}[A]$ counter-probability $\quad (\overline{A} = \Omega \setminus A)$
- $\mathbb{P}\left[\bigcup A_i\right] \leq \sum_i \mathbb{P}[A]$ the *union bound* (a.k.a. Boole's inequality a.k.a. σ-subadditivity)
- $\{A_1, \ldots, A_k\}$ *(mutually) independent* $\iff \mathbb{P}\left[\bigcap_i A_i\right] = \prod_i \mathbb{P}[A_i]$

  An infinite set of events is mutually independent if every finite subset is so.

  k-wise independence means that only all size-k subsets are independent.

Recall: Events

something we can assign a probability to

$A \in \mathcal{F}$ is called an *event* of $(\Omega, \mathcal{F}, \mathbb{P})$; also a *measurable set*.

discrete, $\mathcal{F} = 2^{\Omega}$ (powerset of Ω)

Basic properties

- $\mathbb{P}\left[\overline{A}\right] = 1 - \mathbb{P}[A]$ counter-probability $\quad (\overline{A} = \Omega \setminus A)$
- $\mathbb{P}\left[\bigcup A_i\right] \leq \sum_i \mathbb{P}[A]$ the *union bound* (a.k.a. Boole's inequality a.k.a. σ-subadditivity)
- $\{A_1, \ldots, A_k\}$ *(mutually) independent* $\iff \mathbb{P}\left[\bigcap_i A_i\right] = \prod_i \mathbb{P}[A_i]$

  An infinite set of events is mutually independent if every finite subset is so.

  k-wise independence means that only all size-k subsets are independent.
- *conditional probability* for A given B: $\mathbb{P}[A \mid B] = \mathbb{P}[A \cap B] / \mathbb{P}[B]$
  generally undefined if $\mathbb{P}[B] = 0$
- *law of total probability*: If $\Omega = B_1 \dot\cup B_2 \dot\cup \cdots$ is a partition of Ω, we have

$$\mathbb{P}[A] = \sum_{\substack{i \\ \mathbb{P}[B_i] \neq 0}} \mathbb{P}[A \mid B_i] \cdot \mathbb{P}[B_i].$$

Recall: Random Variables

Random variables (r.v.) $X : \Omega \to \mathcal{X}$; often $\mathcal{X} = \mathbb{R}$ (in general spaces: only *measurable* functions)

Recall: Random Variables

Random variables (r.v.) $X : \Omega \to \mathcal{X}$; often $\mathcal{X} = \mathbb{R}$ (in general spaces: only *measurable* functions)

Basic properties and conventions:

- event $\{X = x\}$ is defined as $\{\omega \in \Omega : X(\omega) = x\}$.
- For event A define the indicator r.v. $\mathbb{1}_A$ via $\mathbb{1}_A(\omega) = [\omega \in A] = \begin{cases} 1 & \omega \in A \\ 0 & \text{else} \end{cases}$

Recall: Random Variables

Random variables (r.v.) $X : \Omega \to \mathcal{X}$; often $\mathcal{X} = \mathbb{R}$ (in general spaces: only *measurable* functions)

Basic properties and conventions:

- event $\{X = x\}$ is defined as $\{\omega \in \Omega : X(\omega) = x\}$.
- For event A define the indicator r.v. $\mathbb{1}_A$ via $\mathbb{1}_A(\omega) = [\omega \in A]$
- $F_X(x) = \mathbb{P}[X \leq x]$ is the *cumulative distribution function (CDF)*.
- for discrete r.v. X define $f_X(n) = \mathbb{P}[X = n]$ the *probability mass function (PMF)*.

Recall: Random Variables

Random variables (r.v.) $X : \Omega \to \mathcal{X}$; often $\mathcal{X} = \mathbb{R}$ (in general spaces: only *measurable* functions)

Basic properties and conventions:

- event $\{X = x\}$ is defined as $\{\omega \in \Omega : X(\omega) = x\}$.
- For event A define the indicator r.v. $\mathbb{1}_A$ via $\mathbb{1}_A(\omega) = [\omega \in A]$
- $F_X(x) = \mathbb{P}[X \leq x]$ is the *cumulative distribution function (CDF)*.
- for discrete r.v. X define $f_X(n) = \mathbb{P}[X = n]$ the *probability mass function (PMF)*.

Independence:

- X and Y independent $\iff \mathbb{P}[X = x \wedge Y = y] = \mathbb{P}[X = x] \cdot \mathbb{P}[Y = y]$ for all x, y.

  (Naturally follows from independent events)

- a sequence of *i.i.d.* r.v. $X_1, X_2, \ldots$ (*independent and identically distributed*) has $X_i \overset{\mathcal{D}}{=} X_1$ and $\{X_i\}_{i \geq 1}$ are mutually independent
  - typical example: sequence of coin tosses (with same coin)

Alternative Random Model

Corollary 1.3

A BST built by inserting n i.i.d. Uniform$(0, 1)$ r.v. has the shape of a random BST.